Team Members and Roles

Navjot Kehal (A01362009) - everything
Repo: https://github.com/Nav-Codes/game-engine
Trello: https://trello.com/b/BkNVvioV/my-trello-board

Engine Design

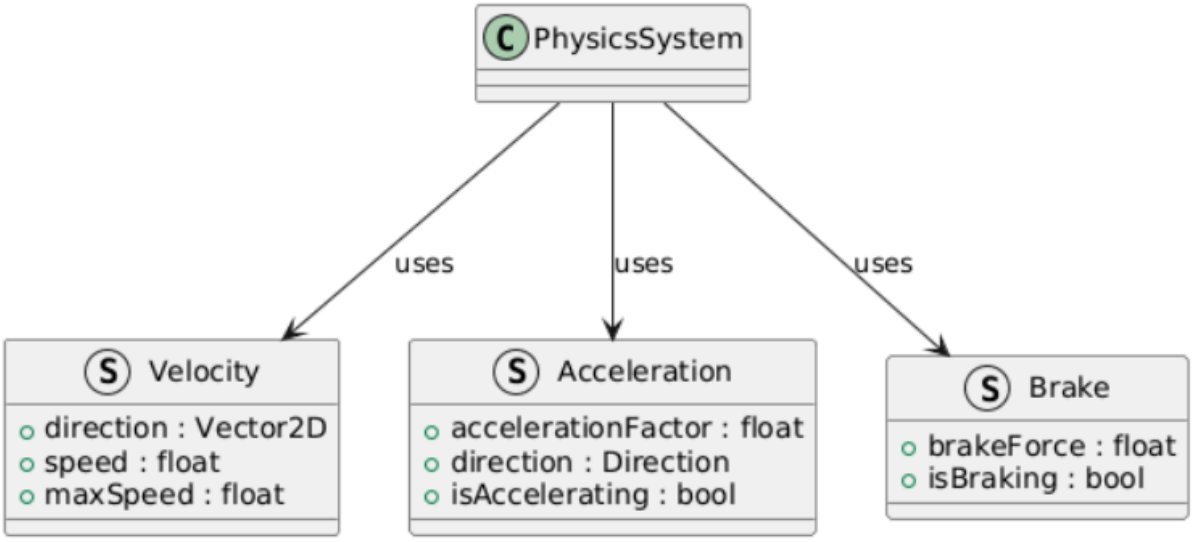
I will be using the lecture material as a foundation for my game engine, which means my game will be 2D (top down) and will also be using SDL3. I will also be using Tiled to create my map because my map system already works with XML files, which will make things easier. There are 6 main systems that will be unique to my game and game engine: physics system, targeting system, keyboard input system, Interaction system and health & damage systems.

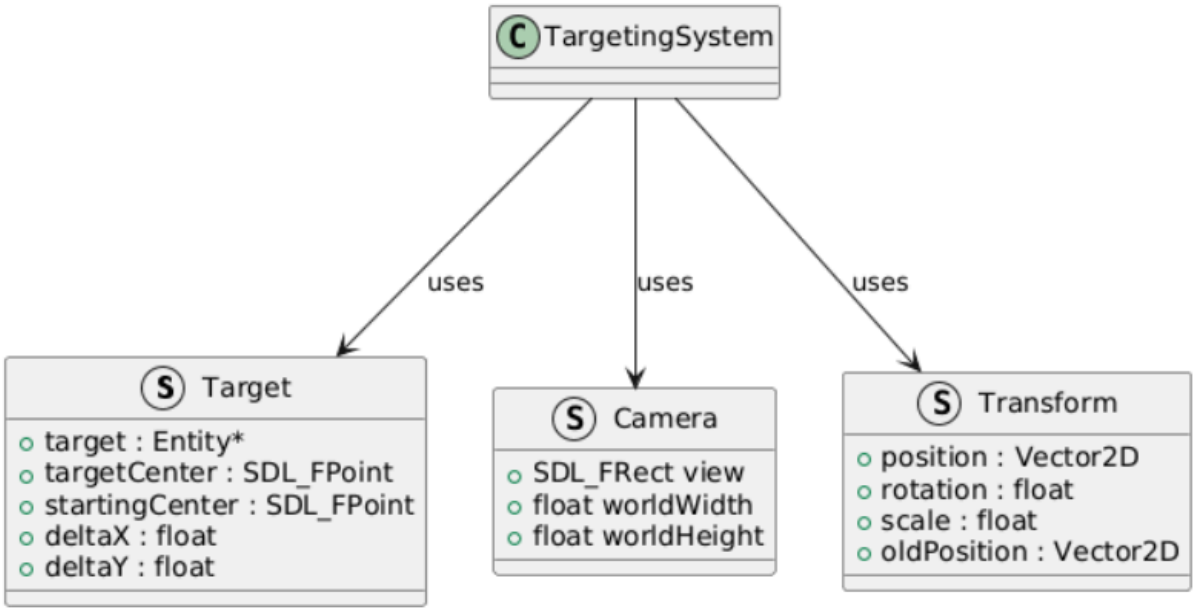
Systems Overview

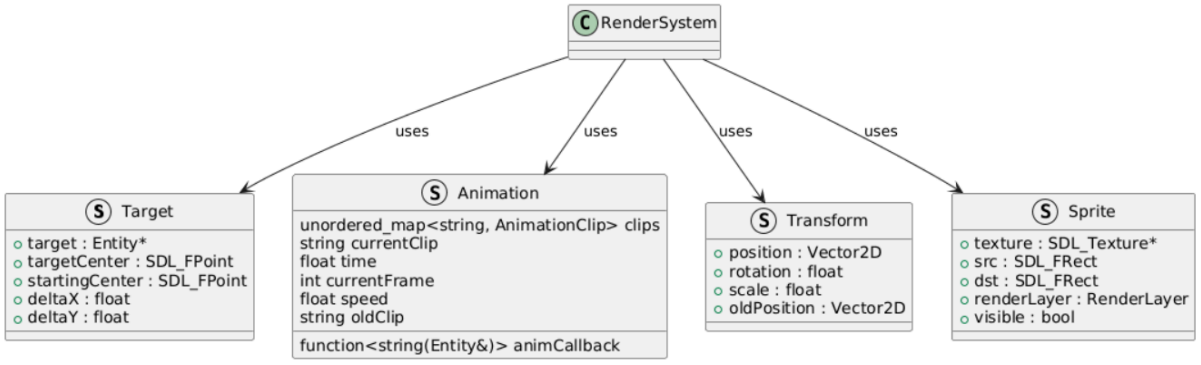
The Targeting System will be the main component that will be used by the player and the enemies. It will ensure that the enemies and the player are aiming in the right direction. The Keyboard Input System now emits actions to the Action Response System, decluttering the Keyboard Input System and minimizing its responsibility. The Action Response System is also another core system that will be used to handle interaction and other user action events, such as getting in and out of the car. The Collision Response System is one of the most important systems in the game. It will help define the actual game logic such as how the player and enemies should take damage, if the player is close enough to the car to get into it, and other game specific logic.

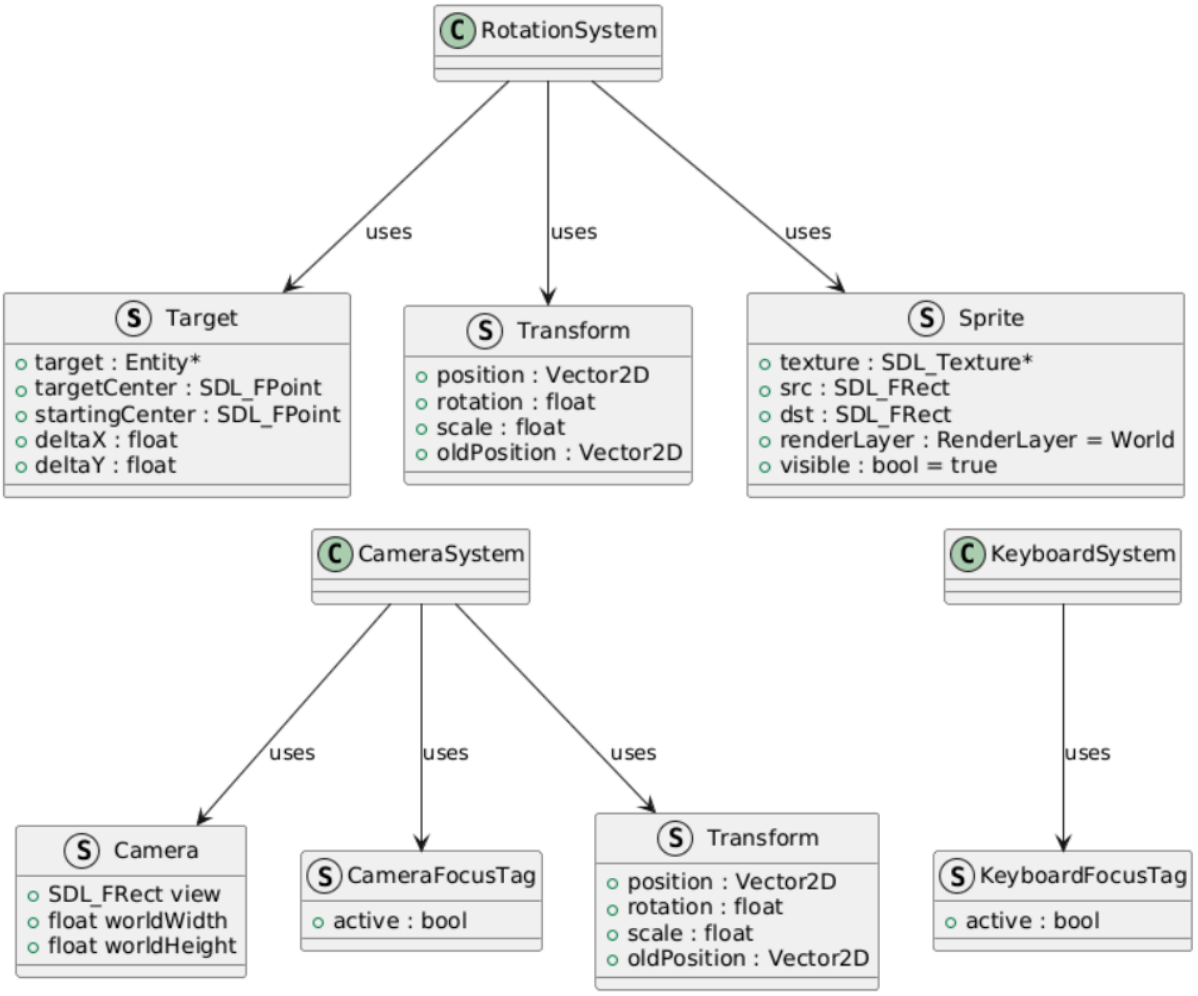
Visual Inspiration

High Level System Diagrams









Systems

Below is a list of all the systems that will be included in the game grouped by visual rendering, general game mechanics, game specific logic and AI:

Visual Rendering

- **Animation system**: Provides a generic way to render sprite animations for different animations. The main idea is that the main animation system will loop through all the entities and will call each entity's respective animation update function. This is because each sprite could have drastically differing XML sheets for their animations or different conditions for how certain animations should be played.
- **Camera system**: Centered on either the player or the car, depending on what the player is controlling. The camera will follow the player/car as they move around and will re center when the player gets in and out of the car.
- **Render system**: Renders the sprites and map onto the scene using SDL3 libraries. Will use data from a Target component that will be modified by the Rotation System to render certain sprites in a specific orientation based on the rotation value that was calculated.
- **PreRender System**: Used to update text if it has changed.
- **HUDSystem**: Updates dynamic text when text requires updating.
- **UIRender System**: Renders the actual text onto the screen only if it has changed.

General Game Mechanics

- **Keyboard system**: Defines how the player can move through the world on foot and in their car. Will contain functionality to swap between on foot and in car controls.
- **Movement system**: Allows the player to move in different directions at varying speeds.
- **Collision system**: Detects collisions with other entities on the map.
- **Destruction system**: Destroys entities that are on the map.
- **Spawn Timer system**: Spawns an entity once every n seconds. Used for spawning in bullets when enemies or players fire their weapon.
- **Event Response System**: Used to handle general action logic such as button clicks.
- **Mouse Input System**: Handles button click events.
- **Main Menu System**: Handles logic for loading the main level.

Game Specific Logic

- **Physics system**: The main purpose of this system is to create a gradual slow down and speed up so something like a car feels like a car when accelerating.
- **360 targeting system**: Handles the firing of projectiles from one point to another, making use of classic high school trigonometry $(y_2-y_1) \div (x_2-x_1)$ to determine the x and y direction of the projectile. Can also be used to determine general distance from one entity to another. Data will be stored in a Target component
- **Rotation System**: Will use the Target component after it is modified by the targeting system to determine a degree angle for rotating sprites.
- **Collision Response System**: When collisions happen, certain game logic actions may need to be performed such as taking damage when a bullet collides with the player. This system can allow for custom game logic for any type of collision.
- **Action Response System**: Allows for specific game logic events to happen when a player performs a certain action such as logic for getting in and out of the car.
- **Spawn Timer System**: Manages projectile creation callbacks during specified time intervals.

Game Design – Mafia Revenge

I will be making a top down shooter with driving mechanics and enemies, all on a large map. The game will be runnable on Windows 11 machines.
Art assets will be sourced from: https://craftpix.net/categorys/sprites/

Story Overview

Mafia is a game about a guy named Tony who seeks revenge against the mafia. He drives around to different areas to kill any mafia members he comes across.

Features

- Tony: AKA the main player; can get in, out and drive a car and can shoot with his gun
    - Health: player can sustain damage from enemies and turrets
        - Health: 10
        - Bullets from enemies deal 1 damage
    - Gun: The player can shoot enemies using their gun

        - Will look the same throughout the game
        - Damage: 1
- Car: Can drive to different locations in a level
    - Driving: Can drive up, down, left and right. Cannot drive in diagonal directions
    - Physics: Gradual acceleration when off and on the gas and braking when holding opposite direction (eg. if holding W, hold S to brake). Can also reverse by holding opposite direction key (eg. if facing north, hold S to reverse)
- Enemies: Shoot in the general direction of the player
    - Health: 3, Damage: 1

Levels

There will only be one level that the player can navigate. This will be a very large level, so players will need to use the car to navigate through the map quickly.

Controls

There are two sets of controls: player movement and car movement

Player

W	A	S	D	E	Mouse Click
Move up	Move left	Move down	Move right	Interact	Shoot

Car

W	A	S	D	E
Accelerate North	Accelerate West	Accelerate South	Accelerate East	Get in/out

List of Required Assets

- **Sprite sheets (animated)**: Player, Enemy, Car.
- **Sprites (non animated)**: Bullet.
- **Map assets**: City map tileset for roads, road lines, sidewalks, houses, shops, buildings, greenery and other props.

MVP

- A mafia guy that moves around and can shoot in 360 degree direction
- Allowing player to get in and out of the car
- Being able to move the car in north, east, south and west directions
- Super basic AI that does not move but just targets player and shoots at them

Tools

- **GitHub**: version control
- **Tiled**: map creation
- **Aseprite**: spritesheet calibration for animations
- **SpriteSheet packer**: Packing multiple sprite sheets and animation clips into one sprite sheet. (https://www.codeandweb.com/free-sprite-sheet-packer)
- **Spritesheet splicer**: Checking size of frames for sprite sheet animations (https://ezgif.com/sprite-cutter)
- **SDL3**: 2D rendering and low level input library
- **CLion**: C++ compiler and IDE

Assets

Below is a list of all the online assets that I sourced for my sprite sheets, map, etc.

- Vehicle sprite sheets: https://minzinn.itch.io/pixelvehicles
- Player & enemy sprite sheets: https://opengameart.org/content/animated-top-down-survivor-player
- Map tileset: https://nyknck.itch.io/citypackpixelart?download
- Title Screen: Generated by ChatGPT

Risks

Car and player hitboxes: There could be a problem when creating the get in/out of car feature because if the car is completely boxed in, the player could potentially spawn on top of the barrier, breaking the game.

- Mitigation: Could make the single hitbox that will be on the car big enough so that when the car hits a barrier, it always leaves enough distance for the player to get out.

Over complicated AI system: Since the enemies, turrets and boss are going to behave differently, it is unclear how much work will be needed to create AI systems for each of these entities.

- Mitigation: Could remove the car system from the game to help with the scope, since the car is just a secondary feature.
- Mitigation: Could make a very basic AI system to account for designing complex car logic.

Map scope creep: Could be too much work to design two unique levels and a boss level in the given time constraints.

- Mitigation: Could just create one large level the player will have to complete. If there is very little time, could just have the starting area and a larger boss level.